

# 2020 Paper

## Question 1

The naive way of computing a value x to the power n is to perform n-1 multiplications. A much better algorithm can involve repeated squaring e.g. x^4 = x —> x^2 —> x^4

Your task is to design a generic algorithm which can compute x^n, where n can be an even or odd value. You must implement your algorithm as a Java class.

There are ways to improve the algorithm, that is, to reduce the number of operations required. For example x^15 can be computed in six steps, but can also be computed more efficiently in five steps.

Your second task is to design a Java program that can be given an integer n and will find (by some form of exhaustive search) the smallest number of multiplication that could be used to raised a value x to the power n.

### First Task

```
public class Solution {
    public static double power(double x, int n)
    {
        // base case
        if (n == 0) {
```

```
            return 1.0;
        }
        double temp = power(x, n / 2);

        // even case for expo
        if (n % 2 == 0) {
            return temp * temp;
        }
        // odd case for expo
        else {
            // postive expo
            if (n > 0) {
                return x * temp * temp;
            }
            //negtive expo
            else {
                return (temp * temp) / x;
            }
        }
    }
    public static void main(String[] args)
    {
        double x = 2.0; // Base
        int n = 15;    // Exponent

        double result = power(x, n);
        System.out.println(x + " ^ " + n + " = " + result);
    }
}
```

The method above uses recursion, the class uses the concept of repeating squaring to efficiently calculate “x” raised to the power of ‘n’, this takes into account both even and odd exponents.

```
public class Solution {
    public static double power(double x, int n) {
        if (n == 0) {
            return 1.0;
```

```
        }

        double result = 1.0;
        if (n < 0) {
            x = 1.0 / x;
            n = -n;
        }

        while (n > 0) {
            if (n % 2 == 1) {
                result *= x;
            }
            x *= x;
            n /= 2;
        }

        return result;
    }
    public static void main(String[] args) {
        double x = 2.0; // Base
        int n = 15;    // Exponent

        double result = power(x, n);
        System.out.println(x + " ^ " + n + " = " + result);
    }
}
```

This method doesn't use the recursion, instead iteration directly follows the logic of repeating squaring algorithm. It calculates the result by iteratively reducing the exponent 'n' and updates the base 'x' accordingly. This method should take less memory and improve the performance especially for larger exponents.

## Second Task: Java Program for Exhaustive Search

```
public class Solution {
    public static void main (String [] args){
        int number = 15;
```

```
        System.out.println("The minumum number of multiplic
ation for " + number
                + "is " + minNumberOfMul(number));
    }
    public int minNumberOfMul (int n) {
        // The question isn't clear enough to me

        if (n <= 1) {
            return 0;
        }
        int minOps = Integer.MAX_VALUE;

        for (int i = 1; i < n; i++) {
            int ops = minNumberOfMul(i) + minNumberOfMul(n
- i) + 1;
            minOps = Math.min(minOps, ops);
        }
        return minOps;
    }
}
```

# Question 2

You are asked to implement in Java an Abstract Data Type (ADT) that can dynamically compute the median age of a changing group of people. We assume that there is a maximum limit to the to the size of the group, **m**. This ADT should implement the following API:

public class MedianGroupAge

| | |
|---|---|
| MedianGroupAge(int m) | Initialises a MedianGroupAge object which can compute the median age of a group of up to m people |
| void addPerson(int age) | Adds the age of a new person in the group |
| void removePerson(int age) | Remove the age of a person from the group |
| Integer getMedian() | Returns the median age of the people currently in the group |

A client of this ADT will perform an arbitrary sequence of calls to the methods of ADT, starting with a call to the constructor. You can assume that getMedian is never called with an empty group of people.

The median of an even number of ages is the average of the two middlemost ages.

Provide an asymptotically optimal implementation of the methods of this ADT. You should describe your implementation, calculate its asymptotic runtime and memory performance, and argue about its optimality.

If you choose, you can use any known ADTs without providing their implementations. However, for each ADT you use you should provide:

- An API of the ADT, similar to the one given above for MedianGroupAge. The API only needs to contain the methods you use in your implementation.
- A brief English description of an implementation of the ADT that has the running time and memory space properties you specify below.
- A tight upper bound of the worst-case asymptotic running time of each of the API methods.
- The memory used by the ADT using the asymptotic notation.

```
import java.util.*;

public class MedianGroupAge {
    ArrayList<Integer> list;
    int size = 0;
    MedianGroupAge(int m) {
        list = new ArrayList<Integer>();
        size = m;
    }
    void addPerson (int age) {
        list.add(age);
    }

    void removePerson(int age) {
        list.remove(age);
    }

    Integer getMedian() {
        list.sort(Comparator.naturalOrder());
```

```
        Integer answer = 0;
        if (list.size() == 0) return 0;
        else if (list.size() % 2==0) {
            int halfway = list.size() / 2;
            int extraNumberEvenCaseNumber = halfway - 1;
            answer = ((Integer) list.get(extraNumberEvenCas
eNumber) + (Integer) list.get(halfway)) / 2;
        }
        else {
            int halfway = list.size() / 2;
            answer = (Integer) list.get(halfway);
        }
        return answer;
    }
    public static void main (String [] args) {
        MedianGroupAge group = new MedianGroupAge(6);
        group.addPerson(20);
        group.addPerson(23);
        group.addPerson(41);
        group.addPerson(27);
        group.addPerson(29);
        group.addPerson(31);

        for (int x = 0; x < group.size; x++) {
            System.out.println("The Median of the List is "
+ group.getMedian());
            System.out.println(group.list.get(x));
        }
        //System.out.println("The Median of the List is " +
group.getMedian());
    }
}
```

Will come back to explain this.

# Question 3

This question is to be answered using Java as the programming language. Design a class hierarchy for a banking environment including details of al member data and al member functions. Fully write the methods which are involved with the computation and withdrawal/lodgement of the interest for al of the account types.

- The bank has many customers. Every customer is identified by name, address and one or more telephone number(s). Customers may have many accounts. Customers may share some accounts with other customers (known as joint accounts).
- Each account is 'owned' by one or two customers, and has a unique account number, a balance and a list of historical transactions.
- Each transaction has a date, amount (positive or negative) and a textual description. Transactions are regarded as affecting the account balance at the end of the current day.
- There are different types of account and each type of account has a specified annual percentage rate (APR). The account types are Current, Savings, Credit Card and Mortgage. Any change in interest rate applies from the following day. Interest is paid (or charged) on every account at the end of the last day of every month.
- To convert the annual percentage rate (APR) to a daily interest rate (DPR)

$$DPR = (1 + (1 + \frac{APR}{100})^{\frac{1}{365}}) * 100$$

- e.g. if APR=4.1%, then 1.041 to the power of 1/365 is 1.00011 which would mean that DPR would be 0.011%.
- A savings account can only have a positive balance. Withdrawals are permitted as long as the balance remains positive. Interest is earned on the balance on this account.
- A mortgage account only has a negative balance. Lodgements are permitted (until the balance is 0). This account type also has details of the property secured on the mortgage along with the suggested current monthly repayment amount. This amount will be changed if the interest rate changes. Interest is paid on the balance in this account.
- A credit card account has a credit limit associated with it. There are two types of withdrawal from a credit card account (cash withdrawals and credit card charges/purchases). Both types of withdrawal from the credit card account

should not be permitted if the credit limit has been reached. Lodgements can be made to a credit card account at any time. Customers are allowed interest free credit on al charges/purchases from the date of the transaction until 10 days after the end of the month. Interest is charged on all cash withdrawals from the date of the withdrawal.

- A current account does not earn interest on positive balances in the account but is charged interest on any negative balances (referred to as overdrafts). This account type has a maximum overdraft amount associated with it (which is the maximum negative balance allowed on the account).